

Tarea 1 - 2024-2 Nicolas Hormann

Parte 1

1- ¿Qué estructuras de datos elegiste para cada consulta y por qué?:

- **Pinguino**: Representa un pingüino.
- **PinguinoList**: Lista ligada para almacenar resultados temporales. Tiempo de inserción $O(1)$.
- **NodoAvl**: Árbol AVL para búsqueda por edad y rango de peso. Complejidad $O(\log n)$.
- **RangeTreeNode**: Árbol de rango para búsqueda en el eje X, con subárboles asociados para Y.
- **AssocTreeNode**: Árbol asociado para consultas en el eje Y, optimizando la búsqueda en 2D.

2. ¿Cómo enfrentaste los retos de consultas geométricas, como las búsquedas en rectángulos o círculos?

- **Consulta en Rectángulo**: Se implementó un árbol de rango, utilizando el nodo de división `vsplit` para recorrer subárboles que contienen pingüinos dentro del rango de coordenadas. Las coordenadas de los pingüinos dentro del rectángulo se ordenan usando la función `qsort`. Creo que esta función me enlentece mucho mi código.

- **Consulta en Círculo**: Realizamos primero una consulta rectangular para delimitar un espacio en el cual quepa el círculo, después el filtro para ver si están o no dentro del círculo.

Nos podríamos haber aproximado a esta pregunta utilizando `partitions` para determinar los pingüinos dentro del rectángulo, y para el círculo ir dividiendo en 4 el plano e ir tomando el cuadrante donde está el círculo para hacer la siguiente partición.

Use la capsula para realizar la parte 1.

Bonus

El Fractional Cascading utilizada para optimizar las búsquedas en árboles de rango. El objetivo es reducir el tiempo de búsqueda al reutilizar resultados de búsquedas previas mediante punteros entre diferentes niveles de la estructura.

Fractional Cascading permite que las consultas geométricas en árboles de rango se realicen en $O(\log n + k)$ en lugar de $O(\log^2 n + k)$. Esto se logra mediante la adición de punteros que enlazan nodos en subárboles asociados, eliminando la necesidad de realizar búsquedas repetitivas en cada nivel. - Ventajas: Reduce significativamente el tiempo de búsqueda, manteniendo la eficiencia espacial.

Intente aplicar el Fractional Cascading pero no fui capaz de hacer la búsqueda correctamente, yo hice primero filtrar con una búsqueda por el eje x y luego otra por el dentro de esos del eje x filtrar los que cumplen con el eje y.

Parte 2

1 Parte 2

1. Describe todas las tablas y funciones de hash utilizadas en tu solución. Explica como manejas el factor de carga para cada caso.

- Tabla principal de hash (`HashTableEntry`): Cada entrada en esta tabla representa un pingüino, identificado por su `penguinId`. La función de hash utilizada es:

$$\text{hash}(\text{penguinId}) = \text{penguinId} \% \text{TABLE_SIZE}$$

Donde `'TABLE_SIZE = 252323'`, un número primo grande que reduce la posibilidad de colisiones. La tabla asocia el id del pingüino con un `CodeTree` de su árbol y con un puntero a la tabla hash de subárboles.

- Tabla hash de subárboles (`SubtreeEntry`): Cada entrada de la tabla hash de subárboles almacena un hash para un subárbol en el `'codeTree'` de un pingüino. Los subárboles se identifican por un hash generado recursivamente con la función `'combineHashes'`. Esta tabla también maneja colisiones y tiene tamaño `'SUBTREE_TABLE_SIZE = 252323'`.

Manejo del factor de carga:

El tamaño de la tabla se ha definido lo suficientemente grande como para minimizar la probabilidad de colisiones. Dado que las tablas utilizan listas enlazadas para manejar colisiones, el factor de carga no se ajusta dinámicamente durante la ejecución. En caso de una colisión le asignamos el siguiente espacio de la tabla hash y lo vinculamos como una lista ligada con un puntero.

2. Explica como implementaste/hubieras implementado incremental hashing en tu solución

El incremental hashing fue implementado en la función `'generateSubtreeHashes'`. Esta función genera de manera eficiente los hashes de subárboles para cada nodo del árbol binario completo, utilizando la función `'computeSubtreeHash'`.

- Se inicia calculando el hash completo del árbol.
- Luego, se recorre el árbol nivel por nivel, generando hashes de subárboles para cada nodo utilizando las posiciones de sus hijos izquierdo y derecho.
- Los hashes se almacenan de manera incremental en la tabla hash del pingüino (`'subtreeHashTable'`) para evitar recalculer los hashes cada vez que se realice una validación o búsqueda.

Esto permite reutilizar los hashes generados previamente en lugar de recalculer desde cero, optimizando la velocidad de búsqueda y verificación.

Para la construcción de todos los subárboles existentes tenemos una función recursiva que va desde abajo hacia arriba sacando todos los subárboles existentes, en el siguiente paso de la recursión se toman como hojas los que eran el penúltimo nivel, y volvemos a iterar cosa de recorrer todos los posibles subárboles que existen. Y así hasta llegar al árbol de altura 2. Luego guardamos en la tabla hash todos estos códigos de los árboles para reutilizarlos y así implementar incremental hashing. Por los tiempos de ejecución que tengo creo que no quedó bien implementado.

3. Indica como manejas colisiones en caso de existir. En otro caso, justificar porque no hay colisiones.

En ambos casos (tabla de pingüinos y tabla de subárboles), las colisiones se manejan mediante listas enlazadas. Si dos entradas tienen el mismo índice en la tabla hash, las entradas colisionadas se enlazan en una lista en esa posición de la tabla.

- En la función `'insertarEnHashTable'`, si hay una colisión al insertar un nuevo pingüino, el nuevo `'HashTableEntry'` se enlaza al inicio de la lista en la posición correspondiente.
- En la función `'storeSubtreeHash'`, si ya existe un hash de subárbol con el mismo valor, se revisa si la posición del subárbol ya está asociada a ese hash. Si no es así, se agrega la posición en la lista correspondiente.

Esto garantiza que las colisiones no causen la pérdida de información y que todas las entradas sean accesibles en tiempo $O(1)$ en promedio, al recorrer la lista en caso de colisiones.

Resumen

En la primera parte use y me base en la capsula, use para la primera funcion un avl simple usando de criterio la edad y como segundo el id. Para la segunda una construccion recursiva del arbol hacia arriba cosa de que queden las hojas del arbol ordenadas de menor a mayor peso y como segundo criterio por id, para asi retornar segun los rangos. Para los rectangulos y circulos trate de usar factorial cascading pero no lo implemente del todo bien entonces quedo en un range tree. El tiempo de ejecucion en los test hard supera los 5 segundos por mucho y nos mantenemos dentro de los paramentos de memoria sin memory leaks. Pero pasamos todos los test cases de manera correcta.

En la segunda parte trate de usar incremental hashing pero parece que no quedo bien implementado porque nos pasamos en los tiempos de ejecucion. No tenemos errores ni fugas de memoria, y tenemos errores en los test cases cuando los arboles son mayores a 15 nodos, Se que no sacamos bien todos los subarboles pero no pude solucionarlo.

Bibliografia:

<https://courses.csail.mit.edu/6.851/spring10/scribe/lec03.pdf>
Capsula de video 1 y 2